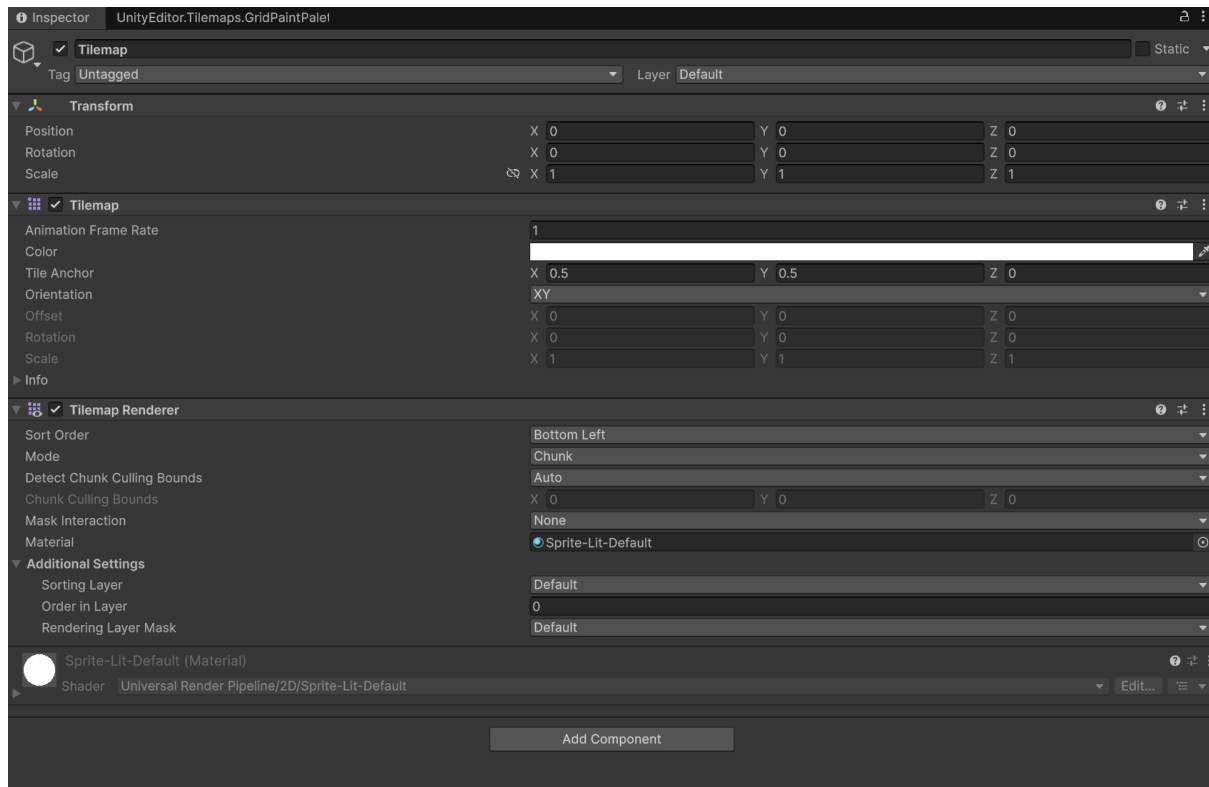


Technical Document 1

Level 1

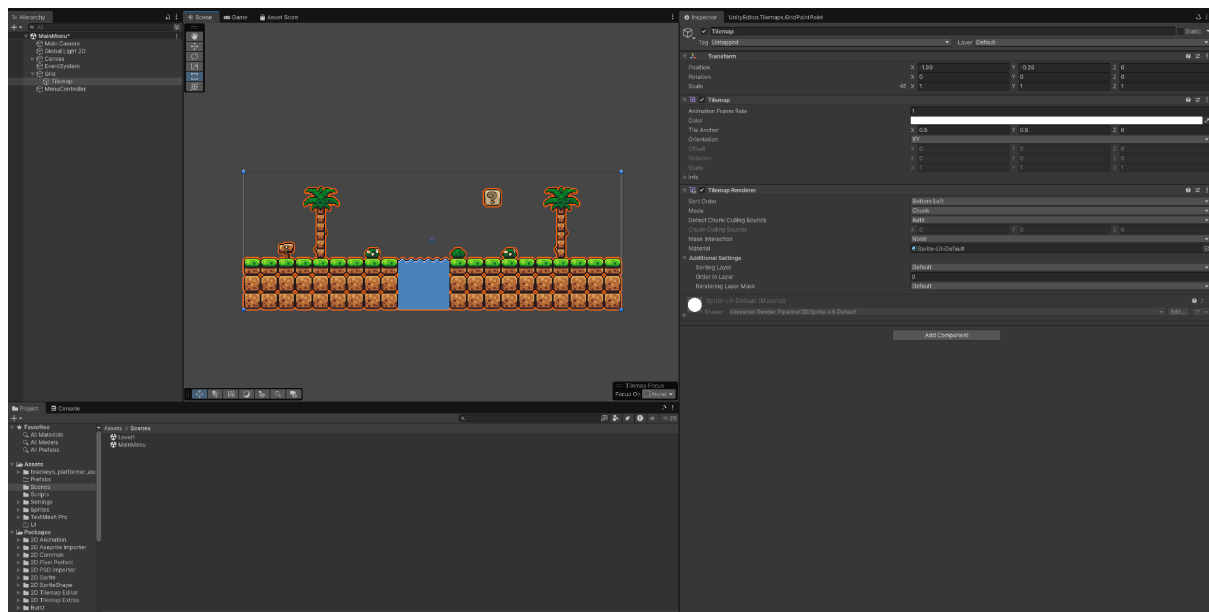


In the image above we can see the game object “*Tilemap*” inspector where we can see the basic component “*transform*” that serves to mark the position, rotation and scale of the game object.

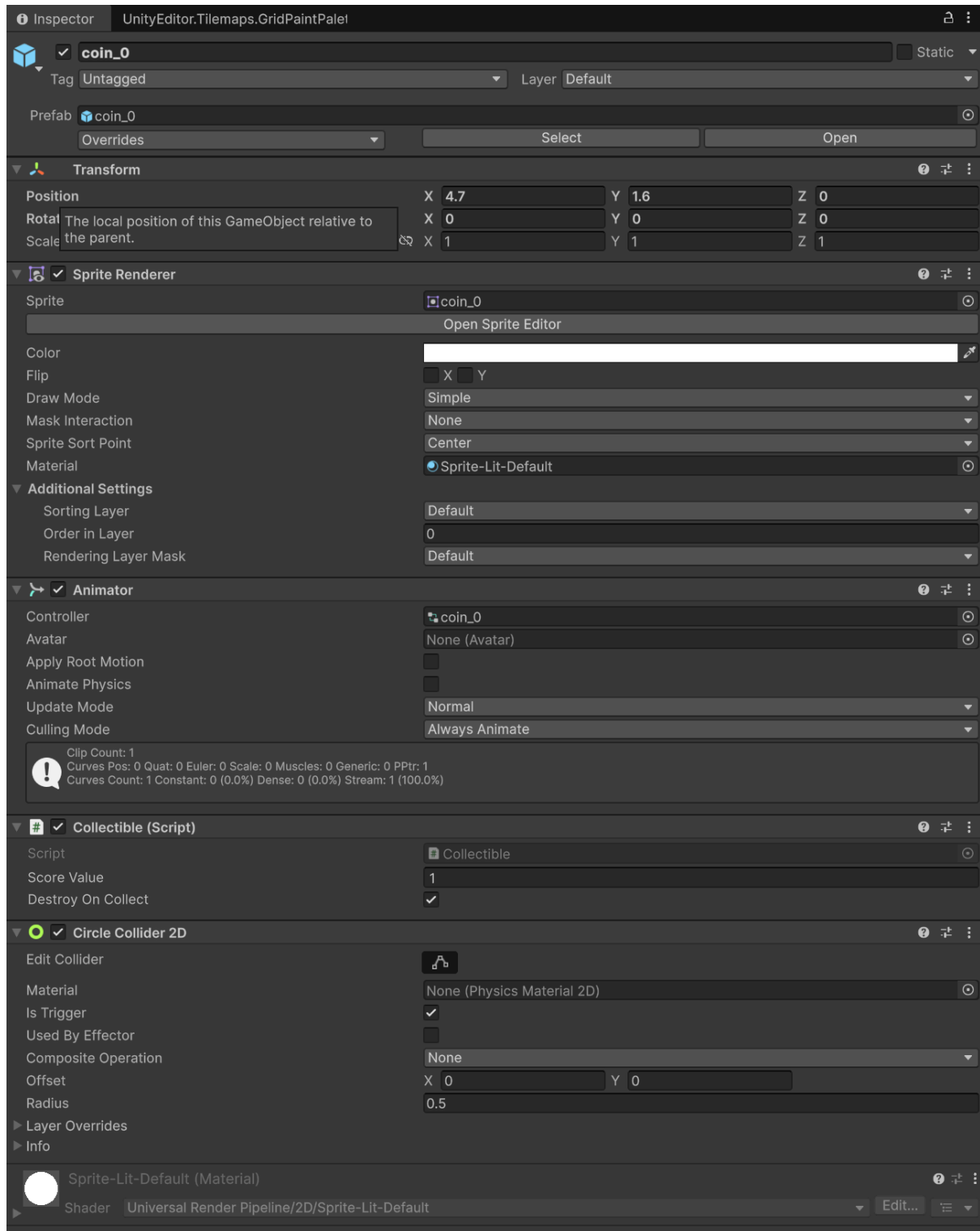
Then we can see the “*Tilemap*” component that has the properties “*Animation Frame Rate*” that defines the speed of frames per second of an animation made with animated tiles, then we have the “*Color*” property that defines the color of the tiles, the “*Tile Anchor*” property defines where the tile stays centered inside the cell grid, the “*Orientation*” property defines the layout where the “*Tilemap*” is, in this case XY which is normal for a 2D game because like the name says it only has 2 dimensions, then we have “*Offset*” property this property moves the position off the tiles relatively to the grid, the “*Rotation*” property just rotates the tiles from the “*Tilemap*”, finally we have the

“scale” that alters the dimensions of the tiles inside the “Tilemap” and “1,1,1” means normal size.

Now I am going to talk about the “Tilemap Renderer” component, this component has these important properties, “Mode” that defines how the tiles are rendered, also “Sort Order” that defines the order from which the tiles are drawn in relation to other objects, then “Material” that defines the shader used to render the tiles, then we have the “Mask Interaction” that defines how the “Tilemap” interacts with sprite masks and to finish we have the “Chunk Culling Bounds” that controls the area used to determine when parts of “Tilemap” can stop being rendered to increase the performance.



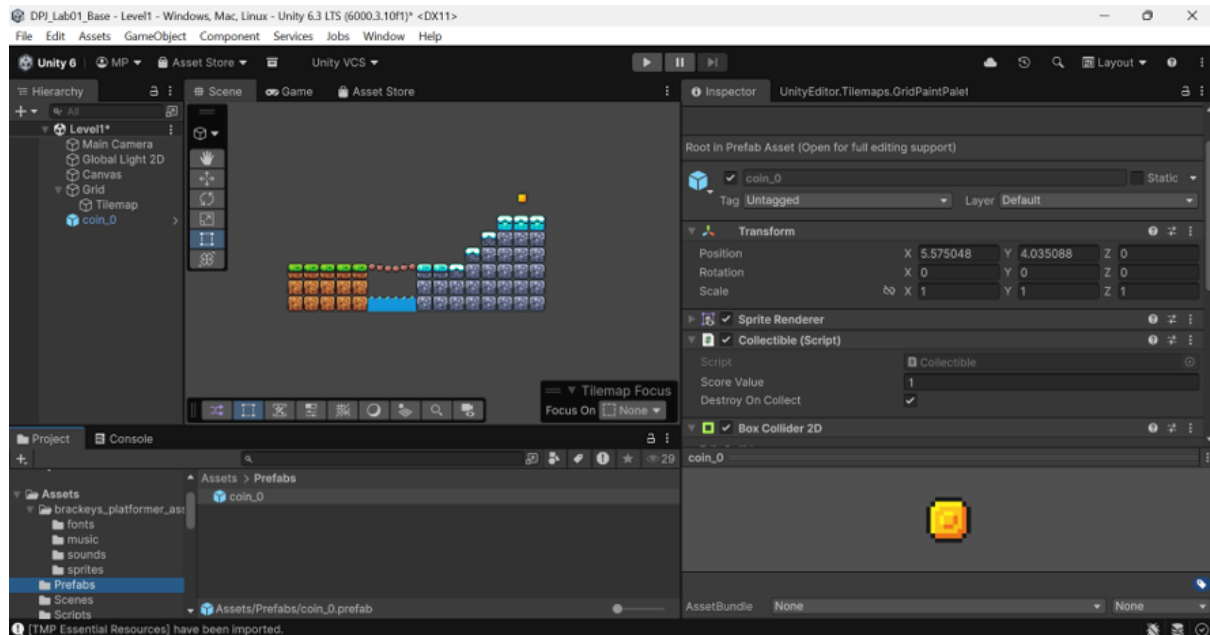
Level 2



Here we have the game object I just created called “coin_0” basically it has the “collectible(script)” that makes it like it says, a collectible that the player can collect and gives a score of 1, also destroys itself when

collected to be a one collect only, then we have the “circle collider 2D” that gives it a hitbox with a 0.5 radius for the player to touch it and activate de “collectible(script)”.

To make sure that the coin is a prefab, we dragged it into the prefabs folder.



```

using UnityEngine;

public class Collectible : MonoBehaviour
{
    public int scoreValue = 1;
    public bool destroyOnCollect = true;

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            if (GameManager.Instance != null)
            {
                GameManager.Instance.AddScore(scoreValue);
            }
            else
            {
                Debug.LogWarning("GameManager não encontrado na cena. Certificar que existe um GameManager com script anexado.");
            }

            if (destroyOnCollect)
                Destroy(gameObject);
            else
                gameObject.SetActive(false);
        }
    }
}

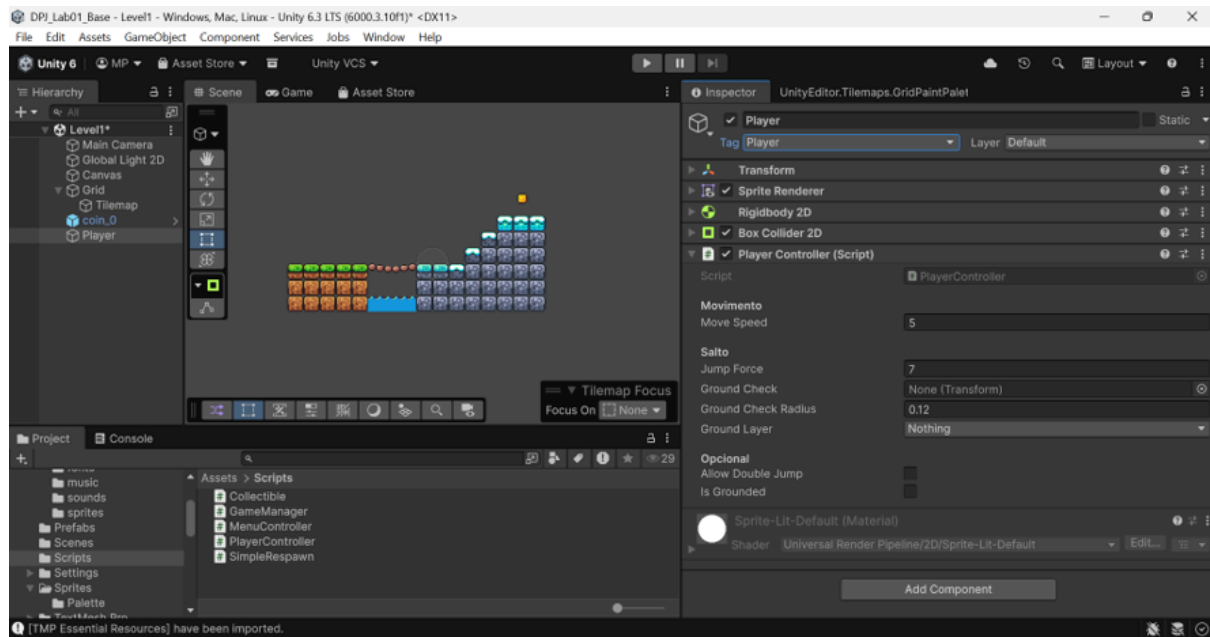
```

By observing the C# code in “[collectible.cs](#)” we can get a vague idea of how it really works:

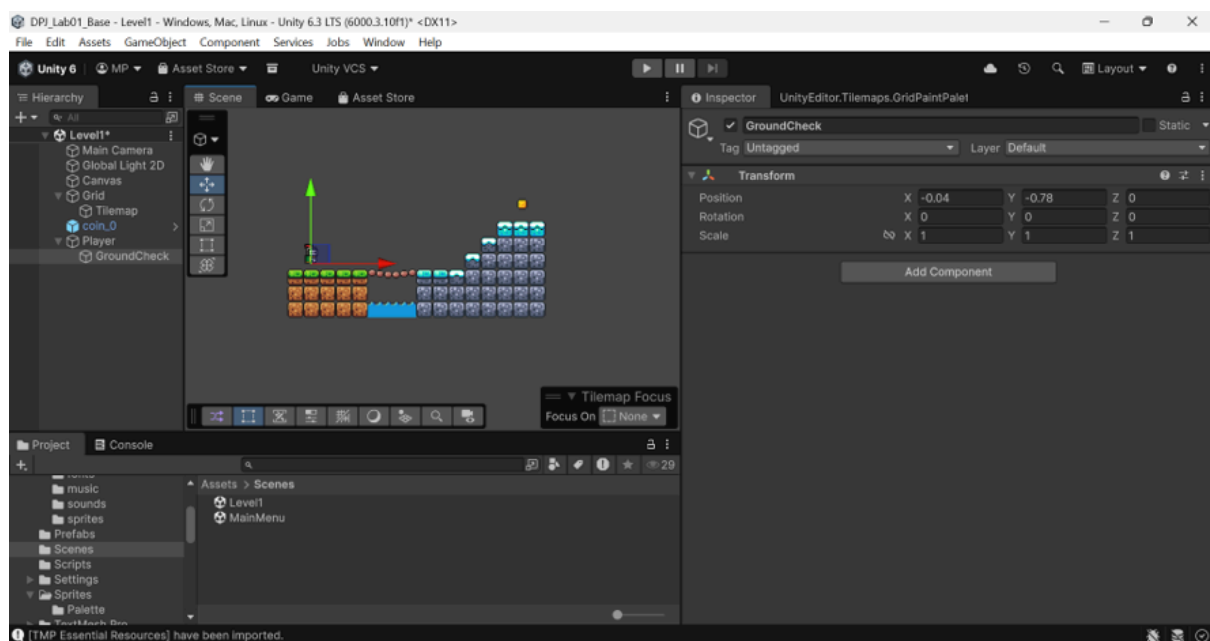
- “[Scorevalue](#)” is how many points the collectible gives when collected;
- The collectible gets destroyed after touching it;
- It then checks if the collision was with the player, and if it is, it adds its value to the player’s score.

Level 3

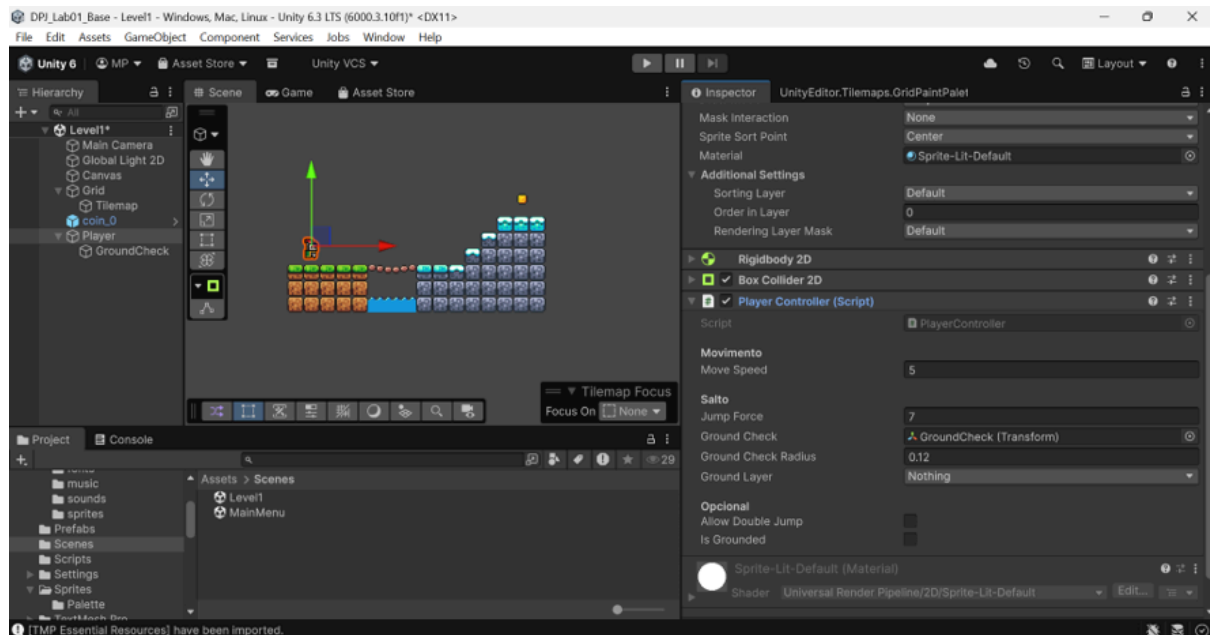
Here we tried creating a player, starting by creating an empty “[GameObject](#)” and placing it in the level, then adding the required components, “[sprite renderer](#)”, “[rigidbody2D](#)”, “[box collider2D](#)”, and the player controller script. Then we tagged the [GameObject](#) as “player” and added a knight sprite to it.



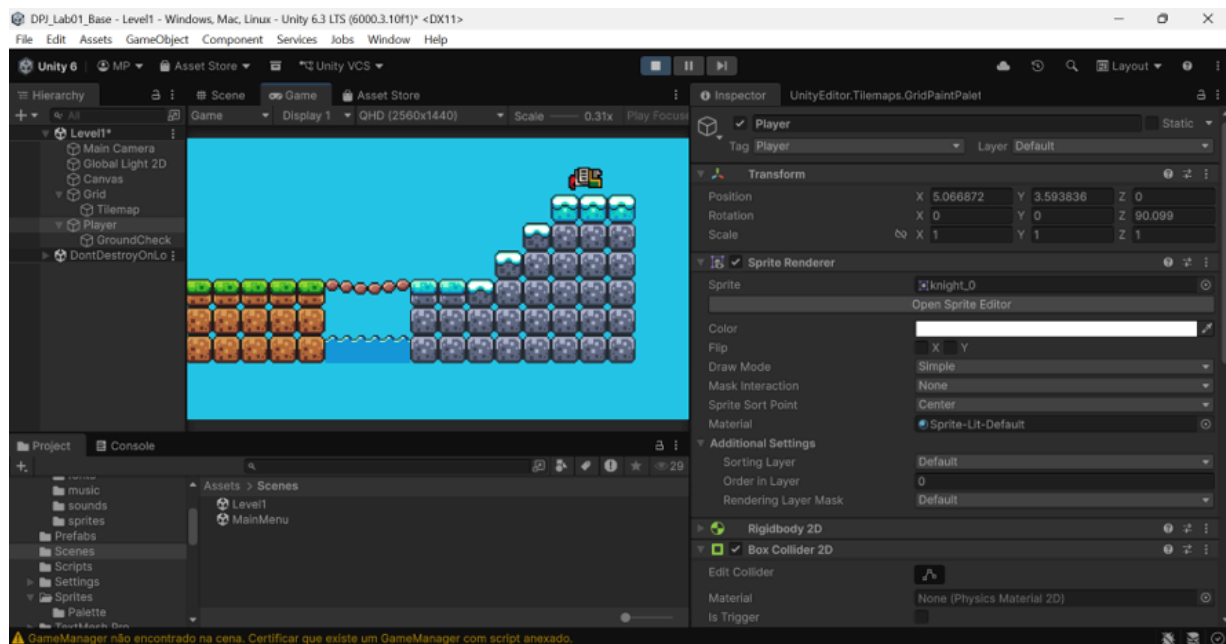
We created a ground check and connected it to the player game object, placing it on the knight's feet so it doesn't fall through the tilemap



After that we made sure to add the ground check object into the player “controller script” through the inspector, just by dragging it into the right place.



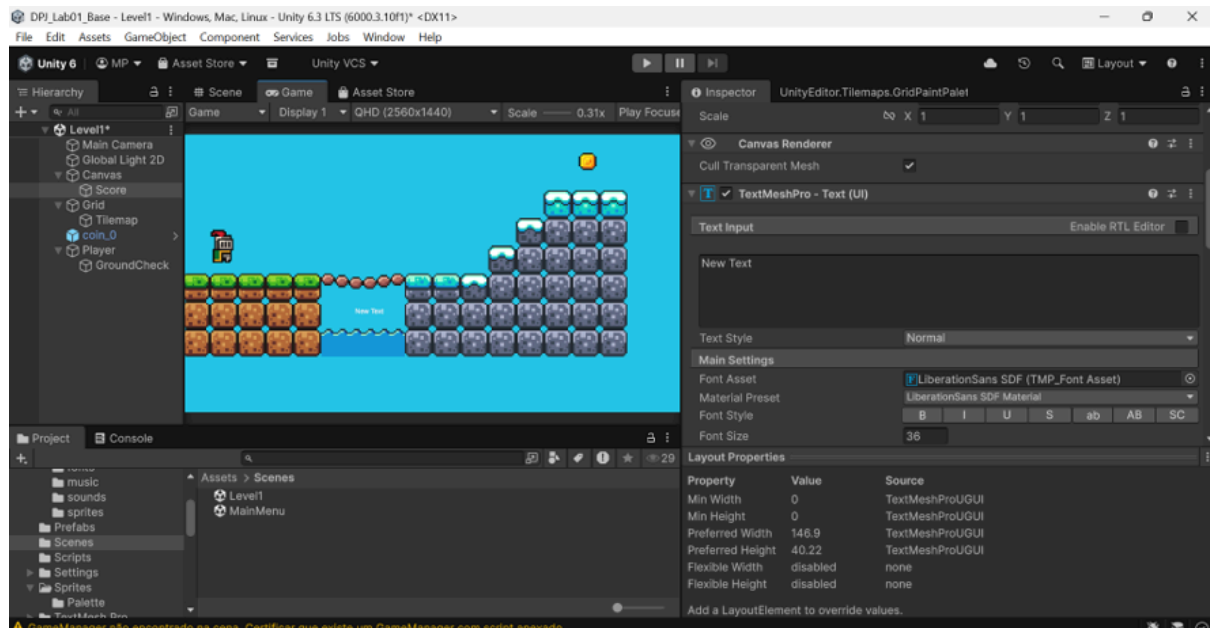
The player fell through the level on our first test, we fixed this by editing the “box collider”, raising its size to correspond with the player, and changing “ground layer” to default.



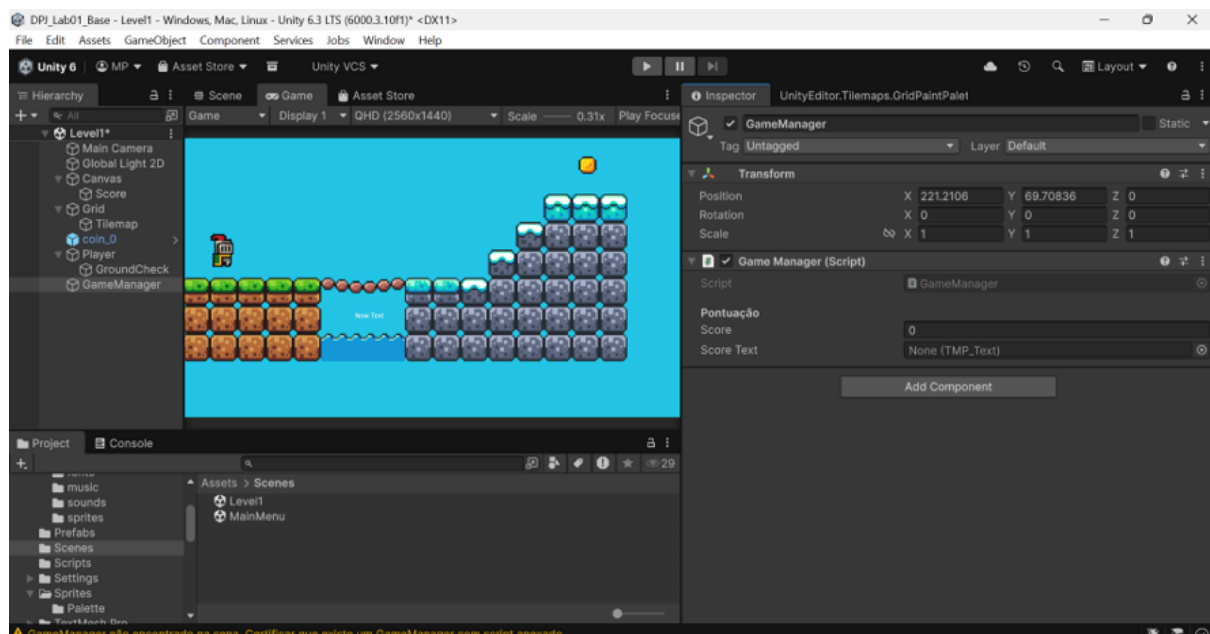
We then managed to make the collisions work, although the player doesn't stay upright...

Level 4

Here is where we make the scoreboard, we started by creating a “TextMeshPro GameObject” inside the canvas and named it score.

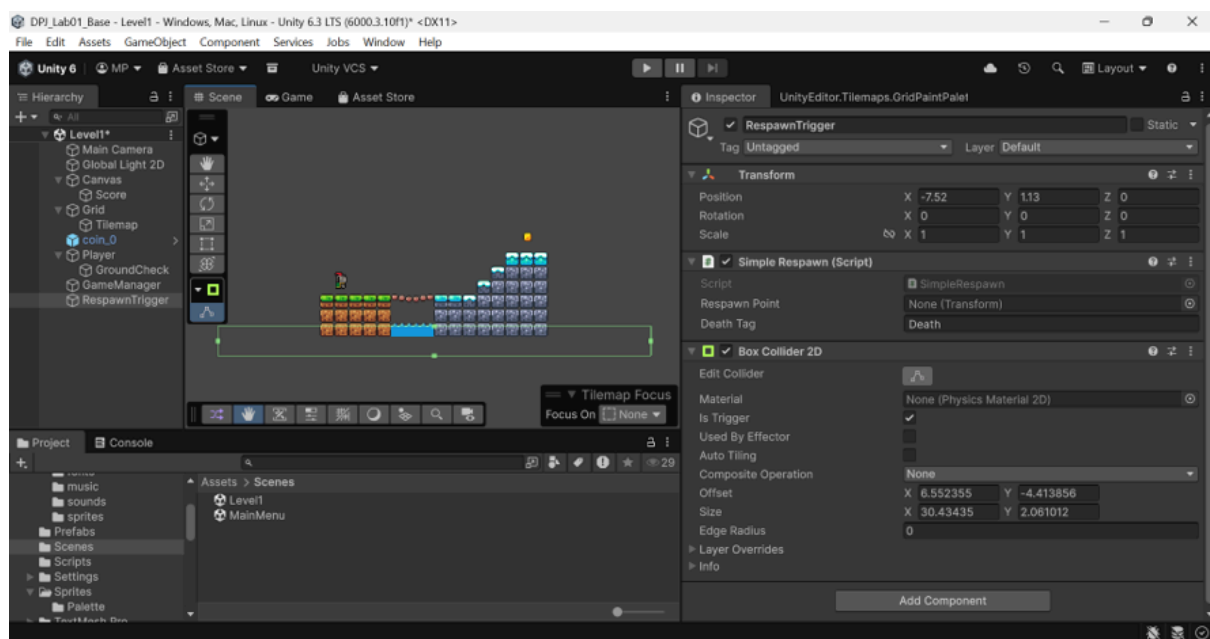
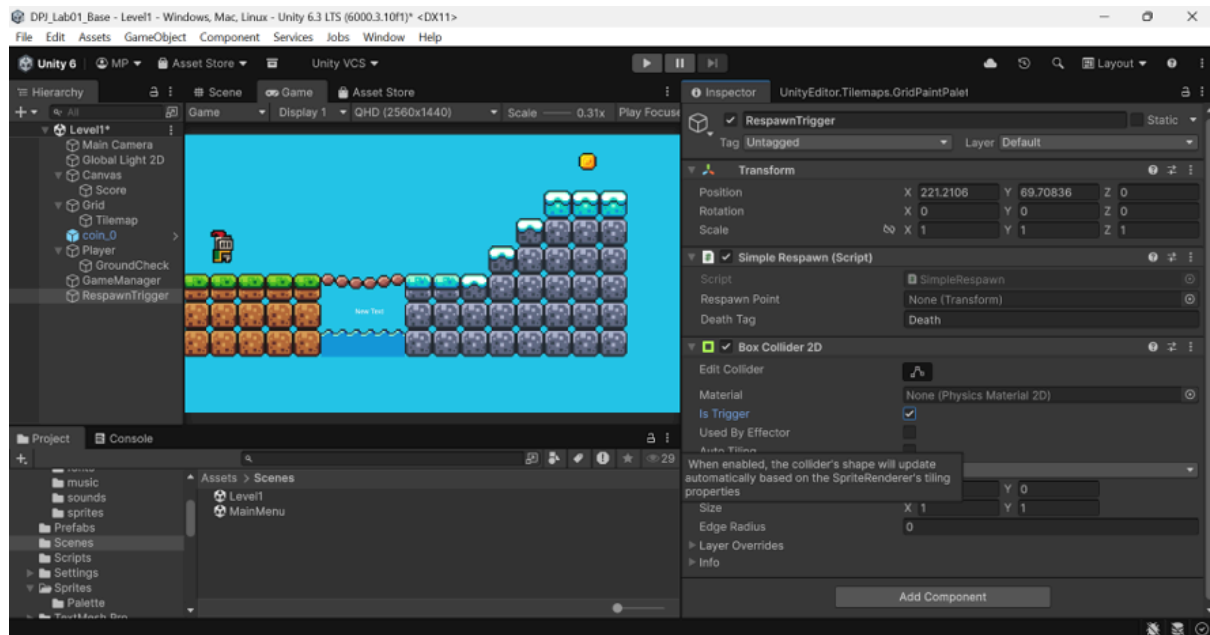


Then we created an empty game object with the “gamemanager script”, we just had to drag the “Score TextMeshPro object” into “Score Text” and it started working as intended



Then we created the “Respawn Trigger GameObject” and the “Respawn Point GameObject”, when touching the respawn trigger it teleports the

player into the respawn point, this is in case the player falls off the map and “dies”.



The respawn trigger needs two components, the simple respawn script and the “BoxCollider2D”, we adjusted the size of the “box collider” to fit widely under the map so the player can’t accidentally jump off without respawning. The respawn point object was placed at the start of the level and it worked automatically, we assume because of the simple respawn script.

After testing the project, we found a few things that we couldn't fix, we first started the game in the main menu scene to test if it goes to the level normally, and it did. The player moves without staying upright, but it doesn't affect the functionality, so we left it as is. After getting the coin and raising the score, the level doesn't restart immediately, only after jumping off the map, but the player is unable to get any more coins since we didn't make them respawn.